

Unit – 1 : Session – 2 : SLO – 2 | Abstraction & Specification

Activity (Solutions)

- 1) `interface Shape { double area(); } class Circle implements Shape { double r; Circle(double r){this.r=r;} public double area(){ return Math.PI*r*r; } }` Use: `Shape s = new Circle(2); System.out.println(s.area());`
- 2) Abstraction via access modifiers: `class Account { private double balance; // hidden data public void deposit(double amt){ if(amt>0) balance += amt; } public double getBalance(){ return balance; } }` — private hides the internal state; public methods expose only controlled operations.
- 3) Real-world analogy: driving a car — the driver uses the steering wheel, pedals and gear lever (public interface) without knowing the engine, transmission or fuel-injection details (private implementation). Similarly, a Java class exposes public methods while hiding fields and internal logic, so users interact with 'what' it does, not 'how' it does it.